

headings

The Impact of Artificial Intelligence on Software Development

OmniLaTeX Research Group

Date: May 2026

Version: 1.0

Classification: Public

Contact: research@omnilatex.dev

The Impact of Artificial Intelligence on Software Development

Artificial intelligence is fundamentally reshaping how software is conceived, built, tested, and maintained. This white paper examines the current state of AI-assisted development, identifies key opportunities and risks, and provides actionable recommendations for engineering leaders navigating this transformation. Drawing on recent industry surveys and case studies, we argue that organizations that strategically adopt AI tools while maintaining strong engineering fundamentals will gain a decisive competitive advantage [chen2024ai].

Contents

Glossaryxix

Symbols **xix**

Numbers **xxi**

Subscripts **xxii**

Acronyms **xxii**

Glossary without page listsxxv

Symbols **xxv**

Numbers **xxvii**

Subscripts **xxvii**

Acronyms **xxviii**

iPreface12Usage3

2.1 Outside tools and special packages **3**

2.2 Using Docker **5**

2.2.1 Compilation 6

2.2.2 More on Docker 11

2.3 Editor **13**

2.3.1 Visual Studio Code 14

2.3.2 Other Editors	17
2.4GitLab	17
3Grundlagen19	
3.1 Wissenschaftliches Arbeiten	19
3.1.1 Ansprüche an die Wissenschaft	20
3.1.2 Recherche	21
3.1.3 Zitieren	22
3.2Typografische Richtlinien	23
3.2.1 Schriftformatierung	23
3.2.2 Befehle	24
3.2.3 Konventionen	24
3.2.4 Einheiten	24
3.2.5 Abkürzungen	27
4Base Features29	
4.1 Fonts and Text	30
4.1.1 Math	32
4.1.2 Sans-Serif	38
4.1.3 Mono-Spaced	38
4.1.4 Symbols	39
4.2Language	40
4.3Sectioning	41
4.3.1 Explanation	41
4.4References	43
4.4.1 Bibliography	43
4.5Lists	47
4.6Censoring	48
4.7Glossaries	49
4.7.1 bib2gls	53
4.8Landscape	55
5Float Features57	

5.1 TikZ and pgfplots	66
5.1.1 Drawing over Bitmaps	66
5.1.2 Trees	66
5.1.3 Flow charts	69
5.1.4 Plotting	69
5.1.5 TikZ and Text	77
5.1.6 Regular TikZ pictures	77
5.1.7 Inkscape	79
5.2Example Boxes	79
6Code Syntax Highlighting83	
6.1 Python	84
6.2MATLAB	84
6.2.1 MATLAB/Simulink icons	86
6.3Modelica	87
6.4Lua	89
An appendix chapter91	
A.1More appendix	91
A.2Unprocessed data	93
BAanother appendix chapter99Index105	

1 Introduction

The software development industry stands at an inflection point. Large language models and AI-powered code generation tools have moved from experimental novelties to production-grade utilities adopted by millions of developers worldwide [microsoft2024copilot]. According to GitHub’s 2024 survey, over 92% of U.S.-based developers are already using AI coding tools in their daily workflows. This white paper synthesizes current research and industry data to provide a balanced assessment of AI’s impact on software development productivity, code quality, and team dynamics.

2 Current Landscape of AI in Software Development

2.1 Code Generation and Completion

AI-powered code completion tools such as GitHub Copilot, Amazon CodeWhisperer, and Cursor have demonstrated significant productivity gains. A controlled study at Microsoft found that developers

using AI assistants completed tasks 55.8% faster than those without, with no significant difference in code quality metrics [peng2023impact].

2.2 Testing and Quality Assurance

AI-driven testing frameworks can automatically generate unit tests, identify edge cases, and predict likely failure points in production code. Organizations reporting early adoption of AI testing tools cite a 30–40% reduction in post-deployment defects [zhang2024testing].

Key Finding

Organizations that combine AI-assisted code generation with AI-driven testing achieve the highest productivity gains—up to 60% faster feature delivery compared to teams using neither tool.

2.3 Documentation and Knowledge Management

Automated documentation tools leverage AI to generate inline comments, API reference documentation, and architectural decision records from codebases, reducing the documentation burden on developers by an estimated 25–35% [patel2024docs].

3 Challenges and Risks

Despite the promising productivity gains, several challenges remain:

- **Code Quality Concerns:** AI-generated code may introduce subtle bugs, security vulnerabilities, or inconsistent patterns that require careful human review.
- **Intellectual Property:** Questions around the licensing of AI-generated code and training data continue to pose legal and ethical challenges.
- **Skill Atrophy:** Over-reliance on AI assistants may erode developers' problem-solving abilities and deep understanding of programming fundamentals.
- **Integration Complexity:** Embedding AI tools into existing CI/CD pipelines, code review processes, and security audits requires significant upfront investment.

4 Recommendations

1. Establish clear guidelines for AI tool usage, including mandatory human review of all AI-generated code.
2. Invest in training programs that strengthen foundational skills alongside AI tool proficiency.
3. Implement robust automated testing pipelines to catch AI-introduced defects early.

4. Monitor and measure the actual impact of AI tools on team velocity, quality metrics, and developer satisfaction.

Key Takeaways

- AI tools can accelerate development by up to 60% when combined with automated testing. Human review remains essential for maintaining code quality and security. Organizations should invest in both AI tools and foundational engineering skills. Early adopters with structured AI adoption strategies gain the greatest competitive advantage.

References

References

- [1] L. Chen, M. Rodriguez, and K. Patel, “The State of AI in Software Engineering: A 2024 Survey,” *IEEE Software*, vol. 41, no. 2, pp. 12–25, 2024.
- [2] Microsoft Research, “GitHub Copilot: Productivity and Quality Assessment,” Technical Report MSR-TR-2024-08, 2024.
- [3] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, “The Impact of AI on Developer Productivity: Evidence from GitHub Copilot,” *arXiv preprint arXiv:2302.06590*, 2023.
- [4] W. Zhang, A. Kumar, and J. Smith, “AI-Driven Software Testing: A Systematic Review,” *ACM Computing Surveys*, vol. 56, no. 4, pp. 1–38, 2024.
- [5] R. Patel and T. Nakamura, “Automated Documentation Generation Using Large Language Models,” in *Proc. ICSE Companion*, pp. 45–52, 2024.